

Author:
Emir Totić

SPRING DATA JPA

Best Practices



Java & Spring lessons
September 2025

WHAT IS DATA JPA

- Spring Data JPA is part of the Spring Data ecosystem that simplifies working with databases.
- It allows us to write repository interfaces instead of manual SQL.
- It automatically translates methods (e.g., findByEmail) into SQL queries.
- It is based on the JPA (Java Persistence API) specification and uses an ORM (e.g., Hibernate).

```
public interface UserRepository extends JpaRepository<User, Long> {  
    User findByEmail(String email);  
}
```

In short: Spring Data JPA = less boilerplate code, faster development, more readable applications.

QUERY DERIVATION

- Spring Data JPA allows us to create methods in repository interfaces without writing SQL queries.
- Method names are analyzed and automatically translated into the corresponding SQL queries.
- This way, we avoid boilerplate code and gain a clean, declarative approach to working with the database.
- The advantage is that the focus remains on business logic, while SQL generation is handled by the Spring Data mechanism.

```
User findByEmail(String email);
```

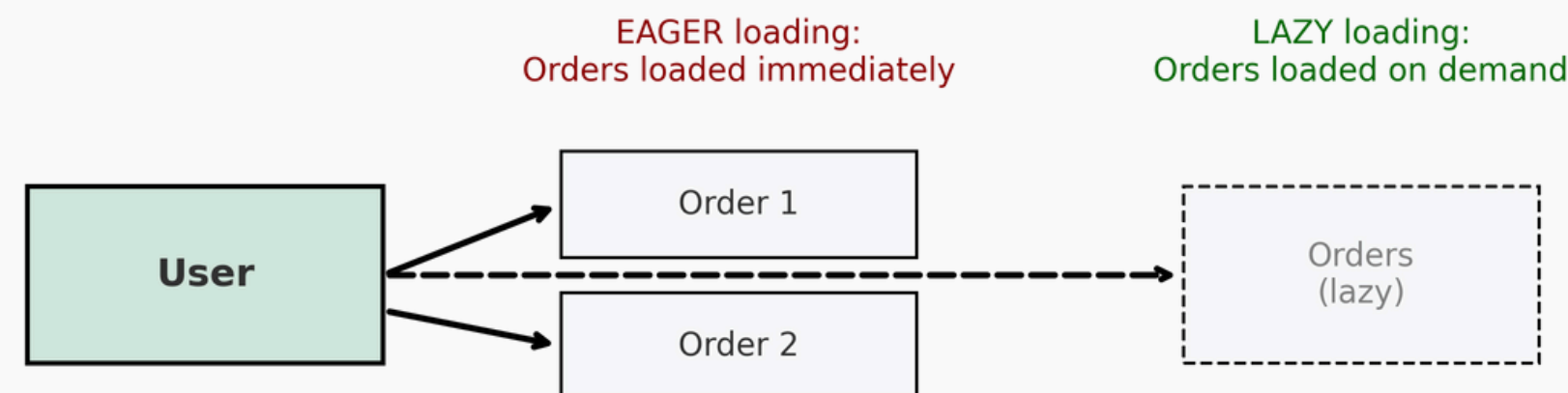
```
List<Order> findByStatusAndDate(String status, LocalDate date);
```

Advantage: readable and intuitive code that clearly expresses the intent of the query.

LAZY VS. EAGER LOADING

- When working with related entities (e.g., User → Orders), Spring Data JPA allows us to choose how the data will be loaded.
- Eager loading: all related entities are loaded immediately with the first query.
 - Advantage: everything is ready right away.
 - Disadvantage: unnecessary database and memory load if the data is not needed.
- Lazy loading: related entities are loaded only when explicitly requested.
 - Advantage: more efficient use of resources.
 - Disadvantage: can cause LazyInitializationException if not used properly.

By default, we use LAZY for collections, and EAGER only when we are certain that the data will always be needed.



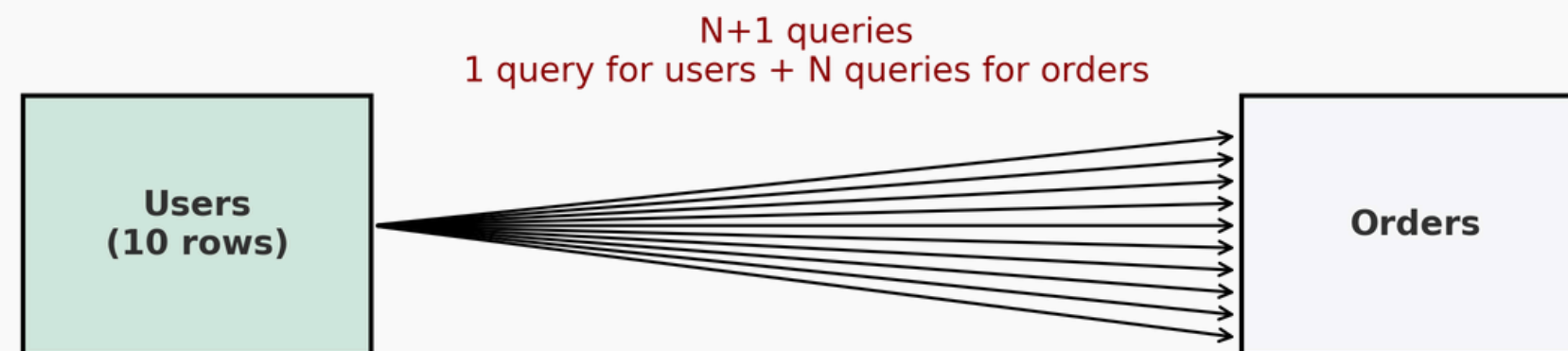
N+1 PROBLEM

- The N+1 problem occurs when the application first executes one query to retrieve a list of entities, and then runs an additional query for each entity to fetch the related data.
- This means: 1 query for the list + N additional queries → a total of N+1.
- The result: heavy database load and slower application performance, especially with large collections.

Example scenario:

- First query: `SELECT * FROM users;` (returns 10 users)
- Then for each user: `SELECT * FROM orders WHERE user_id=?;`

Total: 1 + 10 = 11 queries instead of 1 optimized JOIN.

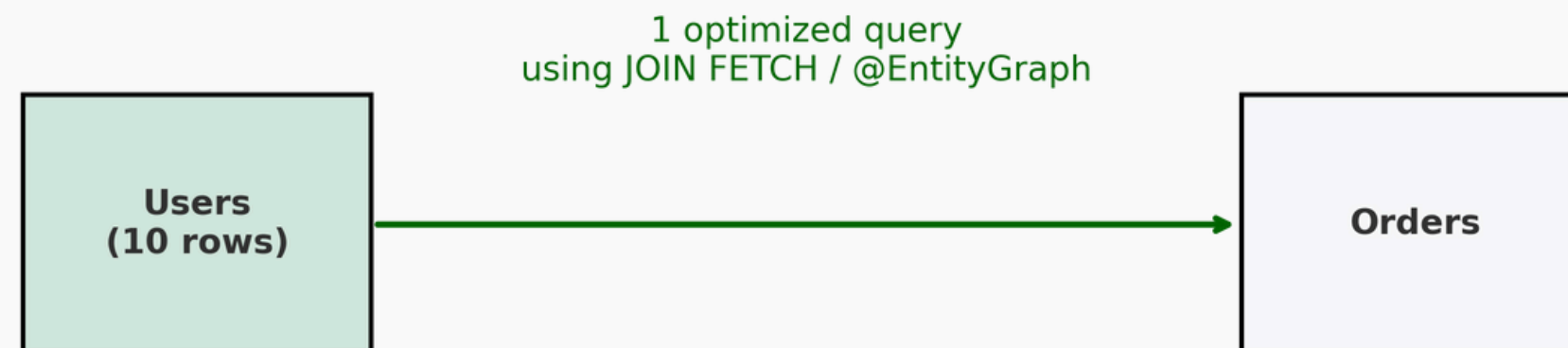


SOLUTION: JOIN FETCH / @ENTITYGRAPH

- The N+1 problem occurs because a separate query is executed for each user to fetch related entities (e.g., orders). This leads to unnecessary database load and slows down the API.
- The solution is to use JOIN FETCH in a JPQL query or the @EntityGraph annotation on repository methods.
- With this approach, Spring Data JPA generates a single optimized query that loads both the main entities and their related data at once.
- In this way, we avoid multiple database calls and significantly improve application performance.

```
@Query("SELECT u FROM User u JOIN FETCH u.orders")  
List<User> findAllWithOrders();
```

Result: returns a list of users with their orders in just one query.



ADVANCED JPA QUERIES

- @Query – uses JPQL (object-oriented query)
- @NativeQuery – allows database-specific SQL
- When to use?
 - @Query: when you want control over the query but remain portable through the ORM
 - @NativeQuery: when optimization or a complex query requires native SQL**

```
@Query("SELECT u FROM User u WHERE u.status = :status")  
List<User> findByStatus(@Param("status") String status);
```

```
@Query(value = "SELECT * FROM users WHERE email = :email", nativeQuery = true)  
User findByEmailNative(@Param("email") String email);
```

ADVANTAGES OF PROPER JPA USAGE

- Less SQL code = faster development

We don't write manual queries for every operation – Spring Data JPA generates them based on repository methods.

- Better performance (no N+1)

By using JOIN FETCH or @EntityGraph we reduce the number of queries and avoid unnecessary database load.

- More readable and declarative code

Query methods clearly express what is being requested (findByEmail, findByStatus) without extra boilerplate.

- Easier application maintenance

When the logic changes, it's enough to update the repository method – the rest of the code stays clean and consistent.

Efficient use of JPA makes the difference between a slow and an optimized API.

CONCLUSION

- Exception handling, query derivation, and fetch strategies are all important aspects of building reliable applications.
- However, optimizing JPA usage goes a step further — it not only improves database interaction but also directly impacts scalability, user experience, and overall system quality.
- By consistently applying best practices, we achieve faster performance, more maintainable code, and cleaner APIs that are easier to extend and integrate with other systems.

Thanks for reading. Let's connect



www.linkedin.com/in/emirtotic



<https://emirtotic.github.io/portfolio-site>



emirtotic@gmail.com